



جامعة دمشق

كلية الهندسة المعلوماتية
قسم الذكاء الاصطناعي
مشروع مادة البرمجة المتوازية

محرك المعالجة عالي الأداء لنظام التجارة الالكترونية

High-Performance E-Commerce Backend Engine

الطلاب:

جورج ميشيل مقبعة جويل إبراهيم زخور
ليث حسان جاويش ميار عبدالله شيخ جوماني
محمد طه محمد غياث صفوه

الطلب الأول: حماية البيانات المشتركة وضمان سلامتها ((Concurrent Access & Data Integrity))

1. توصيف المشكلة ((Problem Description))

في أنظمة التجارة الإلكترونية عالية الأداء (High-Performance E-Commerce Systems)، يتعرض النظام لضغط هائل نتيجة وصول آلاف الطلبات المتزامنة (Concurrent Requests) في جزء من الثانية على نفس الموارد المشتركة (Shared Resources) مما يحدث خلل فيها. في حالتنا هذه تتمثل الموارد المشتركة في:

1. كمية المنتج في المخزون (store_products.quantity): مورد مشترك بين جميع المشترين.
2. رصيد محفظة المستخدم (users.wallet_balance): يعتبر رصيد المستخدم مورد مشترك في حال قام نفس المستخدم بإرسال عدة طلبات شراء متزامنة نقرأ .

عند غياب آليات التنسيق والتحكم بالتزامن (Concurrency Control)، يؤدي هذا الوصول العشوائي المتزامن إلى حدوث تضارب حاد في البيانات وسلوكيات غير متوقعة للنظام (Unpredictable State)، مثل بيع منتجات غير موجودة فعلياً في المخازن (Overselling)، أو السماح بإنشاء طلبات ناجحة دون خصم القيمة الصحيحة من محفظة المستخدم، مما يكسر مبدأ تكامل البيانات وأمانها.

2. سبب المشكلة ((Cause of the Problem))

يعود السبب الجذري لهذه المشكلة التقنية إلى ما يُعرف بال (Race Condition) الناتجة عن نمط القراءة والتعديل ثم الكتابة غير الآمن (Unsafe Read-Modify-Write Pattern).

عبر نطاق معين (بمعالجة عدة طلبات في وقت واحد، فإن نظام التشغيل Apache/PHP-FPM عندما يقوم خادم الويب (عبر نطاق معين) متعددة تعمل بشكل متواز Threads/Processes يوزع هذه المهام على خيوط معالجة (

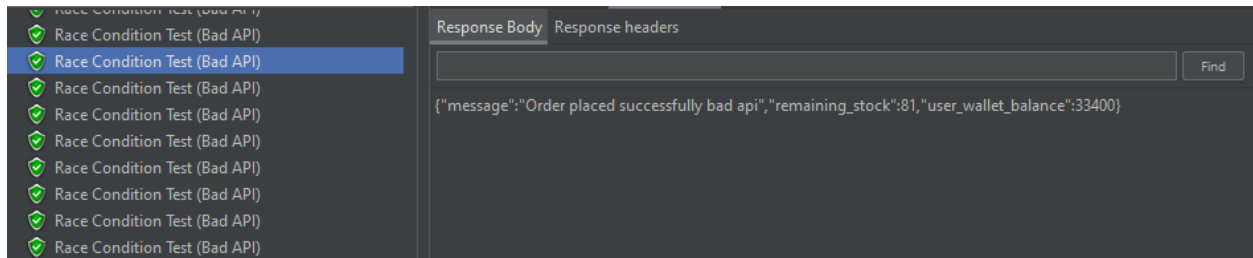
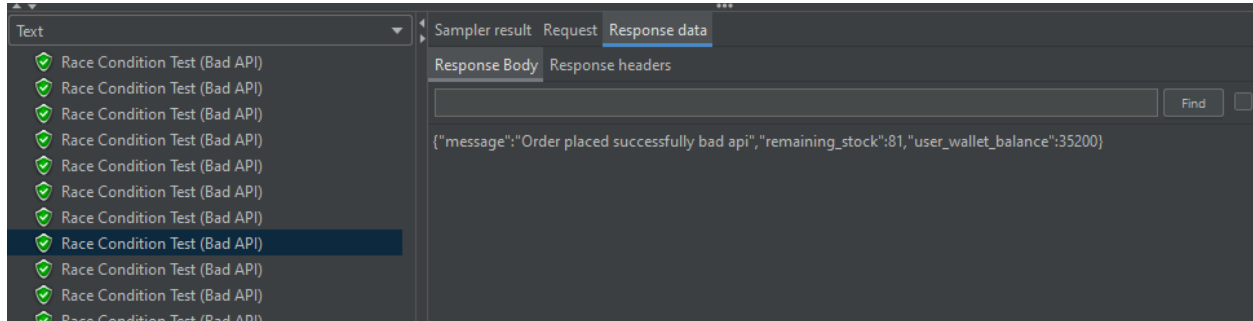
الكود المسبب للمشكلة (Unsafe API):

```
class CheckoutController extends Controller {
  public function checkoutWithRaceCondition(Request $request)
  {
    try {
      $request->validate([
        'user_id' => 'required|exists:users,id',
        'store_id' => 'required|exists:stores,id',
        'product_id' => 'required|exists:products,id',
        'quantity' => 'required|integer|min:1'
      ]);
      $user = User::find($request->user_id);
      $requestedQuantity = $request->quantity;
      $price = Product::find($request->product_id)->price;
      $total_price = $price * $requestedQuantity;
      $storeProduct = StoreProduct::where('store_id', $request->store_id)
        ->where('product_id', $request->product_id)
        ->select('*')
        ->firstOrFail();
      if ($storeProduct->quantity < $requestedQuantity) {
        return response()->json(['error' => 'Not enough stock'], 400);
      }
      usleep(50000);
      $storeProduct->quantity -= $requestedQuantity;
      $storeProduct->save();
      $order = Order::create([
        'user_id' => $user->id,
        'store_id' => $request->store_id,
        'total_price' => $total_price,
        'status' => 'paid'
      ]);
      $user_wallet = User::find($user->id);
      if ($user_wallet->wallet_balance < $total_price) {
        return response()->json(['error' => 'Not enough balance in wallet'], 400);
      }
      $user_wallet->wallet_balance -= $total_price;
      $user_wallet->save();
      return response()->json([
        'message' => 'Order placed successfully bad api',
        'remaining_stock' => $storeProduct->quantity,
        'user_wallet_balance' => $user_wallet->wallet_balance
      ]);
    }
  }
}
```

تظهر المشكلة بالكود باربغ مراحل كما يلي :

- **المرحلة الأولى (قراءة متزامنة):** يقرأ الخيط (Thread A) كمية المنتج الحالية في المخزون وليكن (الكمية = 1)، وفي نفس الأجزاء من الملي ثانية، يقرأ الخيط (Thread B) نفس القيمة (الكمية = 1).
- **المرحلة الثانية (التحقق):** يمر كلا الخيطين من شرط التحقق بنجاح لأن $1 \leq 1$ (الكمية المطلوبة).
- **المرحلة الثالثة (توسيع منطقة الخطر):** تم استخدام الدالة `usleep(50000)` برمجياً لمحاكاة تأخير الشبكة أو عمليات الإدخال والإخراج (I/O Operations)، مما أدى إلى توسيع المنطقة الحرجة (Critical Section) وجعل نافذة التوقيت غير المحددة (Non-deterministic timing) واضحة تماماً للاختبار.
- **المرحلة الرابعة :** يقوم الخيط الأول بطرح الكمية وحفظ المنتج (المخزون = 0)، ثم يقوم الخيط الثاني بفعل الشيء نفسه معتمداً على القراءة القديمة، مما يتسبب في تدمير سلامة البيانات والوقوع في مشكلة الـ **Data Corruption**.

صور من الاختبار توضح المشكلة حيث ان كيف قام طلبين بتحديث نفس الكمية :



3. الحلول المقترحة (Proposed Solutions)

لتحقيق أمان خيوط المعالجة (Thread Safety) وحماية المنطقة الحرجة قمنا بوضع حلين برمجيين أساسيين بناءً على ما قمنا بدراسته في المحاضرات والمقارنة بينهما :

الحل الأول: القفل التشاؤمي (Pessimistic Locking)

- الاستخدام: يُستخدم هذا الحل عند افتراض أن نسبة حدوث التعارضات والتسابق عالية جداً (High Contention).
- آلية العمل: يقوم الخيط الذي يصل أولاً بإغلاق السجل المستهدف في قاعدة البيانات تماماً باستخدام تعليمة `SELECT ... FOR UPDATE` (عبر الدالة `lockForUpdate()`). هذا القفل يمنع أي خيط آخر من قراءة أو تعديل هذا السجل ويجبر بقية الخيوط على الانتظار في حالة حظر (Blocked State) حتى تنتهي المعاملة الحالية (Transaction Commit/Rollback).

الحل الثاني: القفل المتفائل (Optimistic Locking)

- الاستخدام: يُستخدم عندما تكون التعارضات نادرة أو عندما نريد تجنب تكاليف قفل قاعدة البيانات ومنع حدوث اختناقات الأداء الناتجة عن انتظار الخيوط وتبديل السياق (Context Switching Overhead).
- آلية العمل: لا يتم وضع أي قفل أثناء القراءة، مما يتيح لجميع الخيوط القراءة بحرية وبسرعة عالية. ولكن عند مرحلة الكتابة (Write)، يتم التحقق من أن المورد لم يتغير منذ وقت قراءته، ويتم ذلك عبر تتبع رقم الإصدار

(Version Column) إذا فشل التحديث (أي أن خيطاً آخر قد سبقه وعدّل البيانات)، يتم إلقاء استثناء وتراجع المعاملة.

4. الحل المختار والتنفيذ البرمجي (Chosen Solution & Implementatio

n)

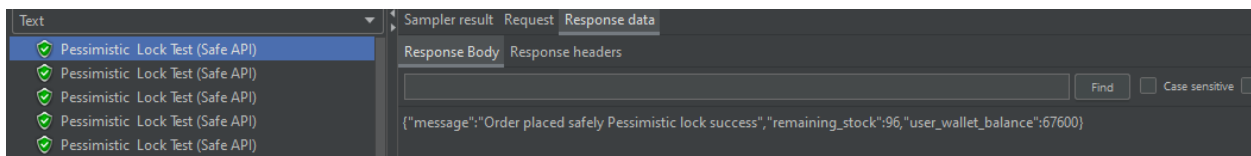
لأغراض المقارنة الهندسية العميقة وتحليل سلوك النظام وتحمل الأداء تم تنفيذ وتجربة كلا الحلين لضمان سلامة العمليات وضمان تحقيق خصائص ACID

أولاً: تنفيذ القفل التشاؤمي (Pessimistic Locking Implementation)

تم عزل العمليات داخل DB::transaction واستدعاء lockForUpdate () لحماية بيانات المستخدم والمخزون بشكل متسلسل :

```
public function checkoutSafePessimistic(Request $request)
{
    try {
        $request->validate([
            'user_id' => 'required|exists:users,id',
            'store_id' => 'required|exists:stores,id',
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);
        $requestedQuantity = $request->quantity;
        $result = DB::transaction(function () use ($request, $requestedQuantity) {
            $user = User::where('id', $request->user_id)->lockForUpdate()->firstOrFail();
            $price = Product::find($request->product_id)->price;
            $total_price = $price * $requestedQuantity;
            if ($user->wallet_balance < $total_price) {
                throw new \Exception("Not enough balance in wallet");
            }
            $storeProduct = StoreProduct::where('store_id', $request->store_id)
                ->where('product_id', $request->product_id)
                ->lockForUpdate()
                ->firstOrFail();
            if ($storeProduct->quantity < $requestedQuantity) {
                throw new \Exception("Not enough stock");
            }
            usleep(50000);
            $storeProduct->quantity -= $requestedQuantity;
            $storeProduct->save();
            $user->wallet_balance -= $total_price;
            $user->save();
            $order = Order::create([
                'user_id' => $user->id,
                'store_id' => $request->store_id,
                'total_price' => $total_price,
                'status' => 'paid'
            ]);
            return [
                'remaining_stock' => $storeProduct->quantity,
                'user_wallet_balance' => $user->wallet_balance
            ];
        });
    } catch (\Exception $e) {
        // Handle exception
    }
}
```

صورة النتيجة من الاختبار:



Text

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

✓ Pessimistic Lock Test (Safe API)

☐ Scroll automatically?

Sampler result

Request

Response data

Response Body

Response headers

Find

```
{
  "message": "Order placed safely Pessimistic lock success",
  "remaining_stock": 0,
  "user_wallet_balance": 10000
}
```

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Pessimistic Loc...	100	15323	7208	19901	3080.22	0.00%	5.0/sec	2.14	1.52	437.9
TOTAL	100	15323	7208	19901	3080.22	0.00%	5.0/sec	2.14	1.52	437.9



نلاحظ هنا ان جميع الطلبات الـ 100 نجحت ولكن نلاحظ ايضا ارتفاع في زمن الاستجابة لان القفل حول الطلبات من متزامنة تدخل بنفس اللحظة الى متسلسلة لا ينفذ طلب حتى ينتهي الطلب السابق من الكتابة

صور الداتا قبل الاختبار وبعد :

	id	name	email	email_verified_at	password	wallet_balance	...
↩ ↪ Edit Copy Delete	1	User One	user1@test.com	NULL	\$2y\$12\$mV.QzhkdC6Y3IEausWXsJuq0u/DoP71ThobRF1pz1e0...	70000.00	N

	id	product_id	store_id	quantity	version	created_at	updated_at
↩ ↪ Edit Copy Delete	1	1	1	100	1	2026-05-18 16:12:05	2026-05-18 16:12:05
↩ ↪ Edit Copy Delete	2	2	1	100	1	2026-05-18 16:12:06	2026-05-18 16:12:06

ثانياً: تنفيذ القفل المتفائل (Optimistic Locking Implementation)

تم الاعتماد على استعلام تحديث شرطي ذكي (Conditional Update) يتأكد من مطابقة الـ version للمخزون ومطابقة القيمة الدقيقة لرصيد المحفظة قبل الحفظ:

```
public function checkoutSafeOptimistic(Request $request)
{
    try {
        @param \Illuminate\Http\Request $request

        $request->validate([
            'user_id' => 'required|exists:users,id',
            'store_id' => 'required|exists:stores,id',
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);
        $requestedQuantity = $request->quantity;

        $result = DB::transaction(function () use ($request, $requestedQuantity) {

            $user = User::findOrFail($request->user_id);

            $price = Product::find($request->product_id)->price;
            $total_price = $price * $requestedQuantity;

            if ($user->wallet_balance < $total_price) {
                throw new \Exception("Not enough balance in wallet");
            }
            $storeProduct = StoreProduct::where('store_id', $request->store_id)
                ->where('product_id', $request->product_id)
                ->firstOrFail();
            if ($storeProduct->quantity < $requestedQuantity) {
                throw new \Exception("Not enough stock");
            }
            $currentWalletBalance = $user->wallet_balance;
            $currentVersion = $storeProduct->version;
            usleep(50000);

            $updatedStock = DB::table('store_products')
                ->where('id', $storeProduct->id)
                ->where('version', $currentVersion)
                ->update([
                    'quantity' => $storeProduct->quantity - $requestedQuantity,
                    'version' => $currentVersion + 1,
                    'updated_at' => now()
                ]);
            if (!$updatedStock) {
                throw new RaceConditionException("Conflict detected on stock.");
            }
            $updatedWallet = DB::table('users')
                ->where('id', $user->id)
                ->where('wallet_balance', $currentWalletBalance)
```

صور النتيجة من الاختبار :

The screenshot shows the REST Client application with two requests. The first request, 'Optimistic Lock Test (Safe API)', is selected and shows a 'Conflict detected' error response. The second request, also 'Optimistic Lock Test (Safe API)', shows a successful response with order details.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Optimistic Lock...	100	4267	962	7132	1739.91	64.00%	13.7/sec	5.79	4.15	431.9
TOTAL	100	4267	962	7132	1739.91	64.00%	13.7/sec	5.79	4.15	431.9

صور الداتا قبل الاختبار وبعد :

id	name	email	email_verified_at	password	wallet_balance	remember_token
1	User One	user1@test.com	NULL	\$2y\$12\$mV.QzhkdC6Y3IEausWXsJqu0u/DoP71ThobRF1pz1e0...	70000.00	N

id	product_id	store_id	quantity	version	created_at	updated_at
1	1	1	100	1	2026-05-18 16:12:05	2026-05-18 16:12:05
2	2	1	100	1	2026-05-18 16:12:06	2026-05-18 16:12:06

بعد:

id	product_id	store_id	quantity	version	created_at	updated_at
1	1	1	64	37	2026-05-18 17:28:33	2026-05-18 17:36:06

id	name	email	email_verified_at	password	wallet_balance	remember_token
1	User One	user1@test.com	NULL	\$2y\$12\$KHsGfN7JS09gm0ZKyUalQeVpnsprjD7k2w1OXI.Oc0e...	48400.00	NULL

5. مقارنة النتيجة قبل وبعد استخدام الحل (Benchmarking & Results))

لإثبات نجاح الحلول تم إجراء اختبار استقرار وضغط (Stress Testing) باستخدام أداة **JMeter** عبر توجيه الطلبات إلى الخادم المحلي المتزامن `ecommerce.test:80`. تم إعداد مجموعة خيوط معالجة (Thread Group) تُحاكي وصول 100 مستخدم متزامن يطلبون نفس المنتج ذو المخزون المحدود (المخزون المتوفر = 100 قطع فقط).

جدول مقارنة السلوك والبيانات:

وجه المقارنة	قبل الحل (API with Race Condition)	بعد الحل (Pessimistic Lock)	بعد الحل (Optimistic Lock)
سلامة المخزون	فاشلة تماماً (Data Corruption) تمت الموافقة على طلبات أكثر من الـ 100 قطع المتاحة، وحدث استنزاف عشوائي للمخزون دون	سليمة بنسبة 100% تم بيع 100 قطع فقط وتصفير المخزون بدقة، ورُفضت بقية الطلبات برمجياً بنتيجة Not enough stock.	سليمة بنسبة 100% تم بيع المنتجات المتاحة بدقة، ورُفضت الطلبات المتعارضة فوراً لحماية integrity البيانات.

	تصفيره بدقة (Overselling). او نقص المخزون بشكل غير منطقي		
سلامة رصيد المحفظة	سمح الخادم بخصومات خاطئة أو قيم سالبة للمحفظة نتيجة التعديل المتداخل وغير المحمي.	تم حماية رصيد المحفظة وعزل العمليات المالية لكل خيط بالكامل دون أي تداخل مالي.	منعت الاستعلامات الشرطية أي تحديث مالي يعتمد على رصيد قديم، وتم حماية أموال المستخدمين.
طبيعة استجابة JMeter	جميع الطلبات الـ 100 تظهر باللون الأخضر (نجاح 200 HTTP)، ولكن بقيم بيانات تالفة داخل قاعدة البيانات!	نجاح الطلبات الـ 100 الأولى بفارق زمني بسيط، وفشل بقية الطلبات مع خطأ (HTTP 400 - Not enough stock).	نجاح الطلبات الأسرع، وفشل الطلبات المتزامنة المتصادمة برمز الحالة (HTTP 409 - Conflict detected) بأمان.
زمن الاستجابة (Latency))	منخفض جداً نظراً لعدم وجود أي انتظار أو حظر للخيط، لكن على حساب دقة البيانات.	مرتفع نسبياً للخيط المتأخرة لأنها تنتظر دورها في طابور القفل التبادلي بقاعدة البيانات (Blocked State).	منخفض جداً وسريع للطلبات الناجحة، وتتحرك الموارد فوراً عند حدوث أول تعارض دون إجمار الخيط على الانتظار.

الطلب الثاني: إدارة الموارد الحاسوبية والتحكم في السعة (Resource Management & Capacity Control)

1. توصيف المشكلة (Problem Description))

في بناء الأنظمة عالية الأداء، لا تقتصر التحديات على حماية البيانات من التضارب فحسب، بل تمتد إلى إدارة الموارد الحاسوبية (Resource Management) مثل المعالج (CPU) والذاكرة العشوائية (RAM). عند تعرض النظام لضغط طلبات متزامن فإن تنفيذ عمليات معقدة أو مستهلكة للموارد بشكل كثيف (Heavy Computations/Memory Allocation) داخل دورة حياة الطلب المباشرة للواجهة (Synchronous Request-Response Lifecycle) يؤدي إلى استنزاف كامل موارد الخادم (Resource Exhaustion/Drain).

تتجلى هذه المشكلة في توقف النظام عن الاستجابة أو انهياره بالكامل وتأخر زمن الاستجابة لبقية الخدمات الحساسة مما يجعل النظام يصطدم بما يُعرف أكاديمياً بـ جدار قابلية التوسع (Scalability Wall)، حيث يعجز الخادم عن تقديم أي مستخدم إضافي بالرغم من أن المشكلة ليست في قاعدة البيانات هذه المرة، بل في نفاذ موارد بيئة التشغيل نفسها.

2. سبب المشكلة (Cause of the Problem))

يعود السبب الجذري هندسياً إلى سوء إدارة **مجمع الخيوط (Thread Pool)** الخاص بخادم الويب بالاشتراك مع سلوك برمجى غير آمن مستهلك للذاكرة والمعالج.

عند وصول طلب إلى الواجهة غير المحمية يتم تخصيص خيط معالجة (**Worker Thread**) من الخادم للتعامل معه. يمر هذا الخيط برحلة تستهلك الموارد كالتالي:

1. **المنطقة الحرجة المقفلة:** يقوم الخيط بحجز قفل تشاؤمي على قاعدة البيانات، وهو سلوك آمن للبيانات ولكنه يضع الخيوط الأخرى في حالة حظر (**Blocked State**).
2. **استنزاف الذاكرة العشوائية (RAM):** مثل عمليات التحميل وإنشاء الفواتير أو الحلقات الضخمة التي تستنزف ال RAM
3. **استنزاف المعالج (CPU Cycles):** تشفير النصوص باستخدام الدالة md5 () نصف مليون مرة لكل طلب يُجبر المعالج على البقاء في حالة الحوسبة المستمرة (**Running State**) واستهلاك كامل دورات المعالج المتاحة، مما يرفع مؤشر ال CPU Usage إلى 100%.
4. **تأخير الخيوط وحجزها:** مثل عمليات ال sleep يحاكي تأخيراً إضافياً يُبقي خيط المعالجة مشغولاً وغير قادر على العودة إلى المجمع (**Thread Pool**) لتخديم طلبات جديدة، مما يرفع من تكلفة تبديل السياق (**Context Switching Overhead**) وينتهي الأمر بنفاد جميع الخيوط المتاحة في الخادم وموتها (**Thread Death**).

الكود المسبب للمشكلة (Resource Drain API):

```

public function checkoutWithResourceDrain(Request $request)
{
    try {
        $request->validate([
            'user_id' => 'required|exists:users,id',
            'store_id' => 'required|exists:stores,id',
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);
        $result = DB::transaction(function () use ($request) {
            $user = User::where('id', $request->user_id)->lockForUpdate()->firstOrFail();
            $storeProduct = StoreProduct::where('store_id', $request->store_id)
                ->where('product_id', $request->product_id)
                ->lockForUpdate()
                ->firstOrFail();
            $price = Product::find($request->product_id)->price;
            $total_price = $price * $request->quantity;
            if ($user->wallet_balance < $total_price || $storeProduct->quantity < $request->quantity) {
                throw new \Exception("Insufficient stock or balance");
            }
            $storeProduct->quantity -= $request->quantity;
            $storeProduct->save();

            $user->wallet_balance -= $total_price;
            $user->save();

            return Order::create([
                'user_id' => $user->id,
                'store_id' => $request->store_id,
                'total_price' => $total_price,
                'status' => 'paid'
            ]);
        });
        $heavyLoadData = [];
        for ($i = 0; $i < 500000; $i++) {
            $heavyLoadData[] = md5($i . time());
        }
        usleep(200000);
        unset($heavyLoadData);
        return response()->json([
            'message' => 'Order placed but system resources were heavily drained!',
            'order_id' => $result->id
        ], 200);
    } catch (\Exception $e) {
        // Handle exception
    }
}

```

Active
Go to S

3. الحل المستخدم وآلية عمله ((The Used Solution & Mechanics))

الحل الهندسي الذي تم اختياره وتطبيقه لحماية الموارد والتحكم في السعة (Capacity Control) هو **تقييد معدل تدفق الطلبات** برمجياً (Throttling / Traffic Rate Limiting) بالتكامل مع ضبط حجم مجمع الخيوط وأدوات المراقبة المتقدمة (Prometheus & Grafana).

آلية عمل : تم دمج طبقة حماية برمجية وبسيطة (Middleware Throttling) تعمل كخط دفاع أول للسياسات الأمنية للنظام. عند تفعيل الـ Middleware، يقوم بفحص هوية العميل المتزامن بناءً على الـ IP أو الـ Token، ويسمح فقط بعبور عدد محدد من الطلبات خلال نافذة زمنية دقيقة جداً (في حالتنا: 50 طلباً في الفترة كحد أقصى throttle:50,1).

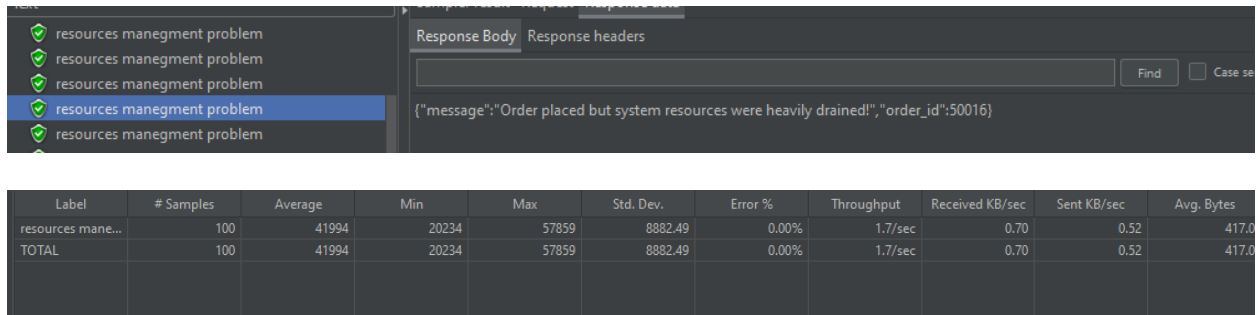
إذا حاول النظام الخارجي (مثل خيوط اختبار JMeter) إغراق الواجهة بطلبات متزامنة تفوق هذه السعة، فإن الـ Middleware يعترض هذه الطلبات الفائضة فوراً في بداية دورة الحياة ويقوم بعملية رفض مبكر وحاسم لها دون السماح لها بالدخول إلى متن الدالة البرمجية المعقدة. هذا يمنع تماماً استدعاء الحلقة التكرارية الكثيفة وعمليات تشفير md5 المستنزفة، ويقوم بإنتاج استجابة سريعة جداً برمز الحالة القياسي HTTP 429 Too Many Requests تحرر خيط

المعالجة ليعود فوراً لمجمع الخيوط (Thread Pool) دون استهلاك أي دورات معالجة حاسوبية أو مساحات ذاكرة عشوائية فائضة.

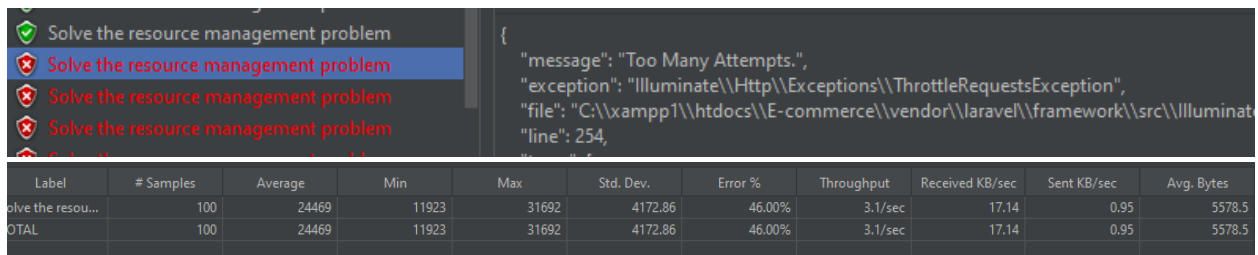
صورة كود الحل:

```
route::post('/checkout/resource-drain', [CheckoutController::class, 'checkoutWithResourceDrain']);
route::post('/checkout/resource-drain2', [CheckoutController::class, 'checkoutWithResourceDrain']->middleware('throttle:50,1');
```

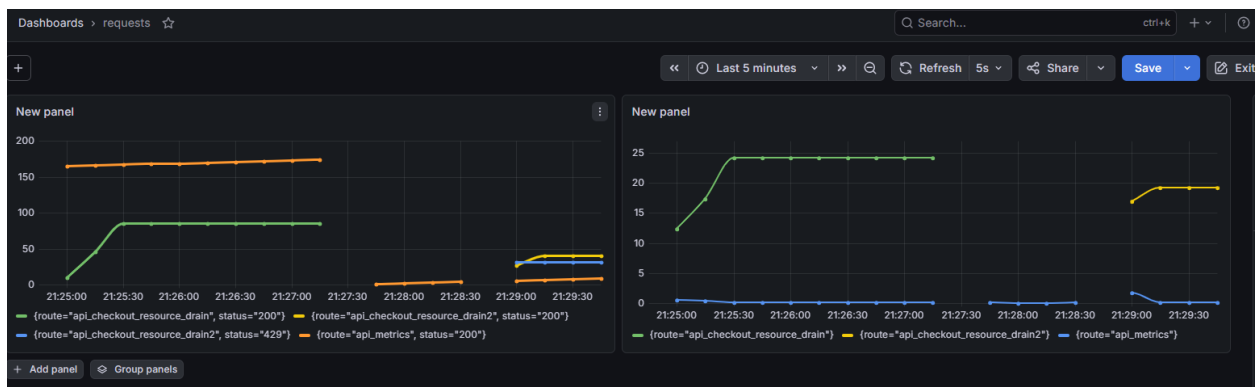
صور الاختبار على Jmeter قبل تقييد معدل تدفق الطلبات برمجياً Throttling / Traffic Rate Limiting وبعد:



بعد :



صور الاختبار على grafana قبل تقييد معدل تدفق الطلبات برمجياً Throttling / Traffic Rate Limiting وبعد:





التحليل العام لنتائج اختبار الأداء والتحكم في السعة

عند إخضاع النظام لاختبار الضغط (Stress Testing) عبر توجيه 100 مستخدم متزامن (100 Threads)، يظهر مؤشر شجرة النتائج سلوك الخادم وحالته الأمنية عبر دلالات الألوان التوضيحية:

- **اللون الأخضر (Green Indicator):** يشير إلى نجاح الطلب شبيكياً واستجابة الخادم برمز الحالة القياسي HTTP 200 OK.
 - في غياب الحماية: يكون المؤشر مضللاً، حيث يعكس نجاحاً ظاهرياً بينما تحدثت كوارث تضارب وفساد في البيانات بالخلفية.
 - في وجود الحماية: يشير إلى الطلبات الأسرع التي عولجت بنجاح وأمان دون انتهاك لقبود النظام.
- **اللون الأحمر (Red Indicator):** يشير إلى فشل أو رفض الطلب من قبل النظام.
 - في الأنظمة المحمية: لا يعبر اللون الأحمر عن انهيار النظام، بل يُمثل خط الدفاع البرمجي الآمن؛ حيث يعكس نجاح طبقة الحماية في صد وإسقاط الطلبات الزائدة فوراً برمز مثل HTTP 429 أو HTTP 409/400، لحماية المعالج والذاكرة وقاعدة البيانات من الاختناق وموت خيوط المعالجة (Thread Death).

جدول مقارنة سلوك النظام (موجز ومُعَمَّم)

وجه المقارنة	الحالة العشوائية ((Baseline))	الحالة المحمية والمقيدة ((Optimized))
دلالة الألوان في JMeter	أخضر متجانس بالكامل (مضلّل).	انقسام التدفق: أولى الطلبات خضراء والبقية حمراء آمنة.
رموز الـ HTTP	تقتصر على HTTP 200 OK.	تتوزع بين HTTP 200 للنجاح، و HTTP 429 / 409 للمرفوض.
سلامة البيانات والموارد	فاشلة: حدوث تضارب مالي وبيع زائد للمخزون.	سليمة 100%: صيانة الموارد وتصفير المخزون بدقة.
استهلاك العتاد (Grafana))	إجهاد كامل للمعالج والذاكرة حتى 100%.	مستقر وآمن، بفضل الرفض المبكر للضغط الفائض.

معدل خطأ مقصود وممنهج لحماية استقرار البنية التحتية.	0% (يُخفي كارثة تقنية خلفية).	معدل الخطأ Error %))
--	-------------------------------	-------------------------

الطلب الثالث: المعالجة غير المتزامنة وإدارة المهام الخلفية (Asynchronous Processing & Background Jobs)

1. توصيف المشكلة (The Bottleneck)

في الكود الأول checkoutWithResourceDrain، تم استخدام نمط المعالجة المتزامنة (Synchronous Execution). في هذا النطاق، يُجبر خيط معالجة الويب (HTTP Worker Thread) على الانتظار حتى تنتهي جميع العمليات، بما فيها العمليات الثقيلة والمستهلكة للموارد (مثل توليد الفواتير، إرسال الإيميلات، أو كما هو موضح في الكود: الحلقة التكرارية المعقدة md5 والتأخير الزمني usleep).

هذا الاحتجاز للخيط (Thread Blocking) يؤدي إلى:

- زيادة هائلة في زمن استجابة الخادم (High Latency).
- استنفاد مجمع الخيوط (Thread Pool Exhaustion) بسرعة، مما يمنع الخادم من استقبال طلبات جديدة.

2. الحل الهندسي المطبق ((The Architectural Solution))

تم حل الاختناق جذرياً بالانتقال إلى نمط المعالجة غير المتزامنة (Asynchronous Offloading) باستخدام طوابير الرسائل (Message Queues) وعمال الخلفية (Background Workers).

يوضح الكود الثاني checkoutAsync هذا التحول المعماري؛ حيث تم فصل العمليات الحساسة والسريعة (تحديث قاعدة البيانات وإصدار الطلب) عن العمليات الثقيلة (الفواتير والمعالجة الحوسبية).

3. آلية العمل ((Engineering Mechanics))

1. الاستجابة الفورية (Fast HTTP Response): يقوم خيط الويب بتنفيذ عملية الخصم المالي والمخزني الآمنة (باستخدام القفل التبادلي lockForUpdate) في أجزاء من الملي ثانية.
2. الترحيل (Offloading): بدلاً من تنفيذ المعالجة الثقيلة، يقوم النظام بتغليف هذه المهمة داخل كائن ProcessOrderBilling وإرسالها إلى طابور الانتظار (Queue) عبر الدالة dispatch().
3. تحرير الموارد (Resource Release): بمجرد إرسال المهمة للطابور، يُرجع خيط الويب استجابة HTTP OK 200 للعميل فوراً، متحرراً ليعود إلى مجمع الخيوط لتخديم مستخدم آخر.
4. عامل الخلفية (Queue Worker): في الخلفية، وبشكل منفصل تماماً عن خيوط الويب، يقوم خيط معالجة آخر (Worker) بسحب المهمة من الطابور وتنفيذ الحلقة التكرارية الثقيلة md5 دون أي تأثير على أداء واجهة برمجة التطبيقات

مقتطفات من الكود : كود المشكلة هو نفس الكود الموجود بالطلب الثاني وهذا كود الحل :

```
class ProcessOrderBilling implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public Order $order;

    public function __construct(Order $order)
    {
        $this->order = $order;
    }

    public function handle(): void
    {
        $heavyLoadData = [];
        for ($i = 0; $i < 500000; $i++) {
            $heavyLoadData[] = md5($i . time());
        }
        usleep(200000);
        unset($heavyLoadData);
    }
}
```



```

public function checkoutAsync(Request $request)
{
    try {
        $request->validate([
            'user_id' => 'required|exists:users,id',
            'store_id' => 'required|exists:stores,id',
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);

        $result = DB::transaction(function () use ($request) {
            $user = User::where('id', $request->user_id)->lockForUpdate()->firstOrFail();
            $storeProduct = StoreProduct::where('store_id', $request->store_id)
                ->where('product_id', $request->product_id)
                ->lockForUpdate()
                ->firstOrFail();

            $price = Product::find($request->product_id)->price;
            $total_price = $price * $request->quantity;

            if ($user->wallet_balance < $total_price || $storeProduct->quantity < $request->quantity) {
                throw new \Exception("Insufficient stock or balance");
            }

            $storeProduct->quantity -= $request->quantity;
            $storeProduct->save();

            $user->wallet_balance -= $total_price;
            $user->save();

            return Order::create([
                'user_id' => $user->id,
                'store_id' => $request->store_id,
                'total_price' => $total_price,
                'status' => 'paid'
            ]);
        });

        ProcessOrderBilling::dispatch($result);

        return response()->json([
            'message' => 'Order placed successfully. Billing is being processed in the background.',
            'order_id' => $result->id
        ]);
    } catch (\Exception $e) {
        return response()->json([
            'message' => $e->getMessage()
        ], 500);
    }
}

```

صور لتنفيذ الاختبار على Jmeter:

The screenshot shows the JMeter GUI. On the left, a tree view contains multiple 'Asynchronous Processing & Background Jobs' elements. The right pane shows the 'Response data' tab for a selected element, displaying a JSON response: `{"message": "Order placed successfully. Billing is being processed in the background.", "order_id": "50085"}`. Below this, a summary table provides performance metrics.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Asynchronous ...	100	16558	7339	19042	2431.62	0.00%	5.2/sec	2.22	1.54	434.0
TOTAL	100	16558	7339	19042	2431.62	0.00%	5.2/sec	2.22	1.54	434.0

صورة التنفيذ بعد المعالجة :



نلاحظ ان زمن الاستجابة انخفض كثيرا عن ما كان عليه قبل اضافة ال worker :

```
C:\xampp1\htdocs\E-commerce>php artisan queue:work
2026-05-18 19:03:47 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:52 App\Jobs\ProcessOrderBilling ..... 4s DONE
2026-05-18 19:03:52 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:53 App\Jobs\ProcessOrderBilling ..... 1s DONE
2026-05-18 19:03:53 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:54 App\Jobs\ProcessOrderBilling ..... 924.65ms DONE
2026-05-18 19:03:54 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:55 App\Jobs\ProcessOrderBilling ..... 865.13ms DONE
2026-05-18 19:03:55 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:56 App\Jobs\ProcessOrderBilling ..... 807.64ms DONE
2026-05-18 19:03:56 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:58 App\Jobs\ProcessOrderBilling ..... 1s DONE
2026-05-18 19:03:58 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:03:59 App\Jobs\ProcessOrderBilling ..... 1s DONE
2026-05-18 19:03:59 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:04:00 App\Jobs\ProcessOrderBilling ..... 869.6ms DONE
2026-05-18 19:04:00 App\Jobs\ProcessOrderBilling ..... RUNNING
2026-05-18 19:04:01 App\Jobs\ProcessOrderBilling ..... 1s DONE
```

4. جدول المقارنة الفنية (المتزامن مقابل غير المتزامن)

وجه المقارنة	checkoutWithResourceDrain (المعالجة المتزامنة)	checkoutAsync (المعالجة غير المتزامنة عبر ال Queues)
زمن الاستجابة للتعامل (Latency))	مرتفع جداً (ينتظر العمل حتى انتهاء الحلقة التكرارية الثقيلة بالكامل).	منخفض جداً (يستجيب الخادم فوراً بعد تسجيل الطلب في قاعدة البيانات).
حالة خيط معالجة الويب (HTTP Thread)	محجوز ومحظر (Blocked) طوال فترة المعالجة الثقيلة، مما يقلل السعة.	مُحرر فوراً (Released) وجاهز لاستقبال طلبات المتجر الأخرى.
الإنتاجية (Throughput))	منخفضة (يصطدم النظام بجدار السعة بسرعة تحت الضغط).	مرتفعة جداً (قدرة هائلة على استيعاب آلاف الطلبات المتزامنة).
عزل الموارد (Resource Isolation)	تستهلك الواجهة الأمامية كامل موارد الـ CPU والـ RAM، مما يهدد استقرار الخادم.	تم عزل إجهاد المعالج والذاكرة في بيئة (Background Worker) يمكن تحجيمها بشكل مستقل.

الطلب الرابع: معالجة البيانات الضخمة على دفعات (Batch Processing)

1. توصيف المشكلة (The Memory Exhaustion Problem)

في أنظمة التجارة الإلكترونية التي تخدم آلاف الطلبات، تتراكم البيانات بسرعة هائلة (مئات الآلاف أو ملايين السجلات يومياً). في الكود الأولي (calculateDailySalesBadApi)، تم استدعاء دالة get() لجلب جميع طلبات اليوم دفعة واحدة وتخزينها في مصفوفة داخل الذاكرة العشوائية (RAM) لمعالجتها.

هذا النهج يحمل خطأً معمارياً فادحاً يتمثل في التعقيد المكاني $O(N^2)$ ؛ حيث يزداد استهلاك الذاكرة بشكل خطي مع زيادة عدد الطلبات. إذا احتوى اليوم على مليون طلب، فإن الخادم سيحاول تحميل المليون سجل (Objects) في الذاكرة دفعة واحدة، مما يؤدي حتماً إلى امتلاء الذاكرة المخصصة للـ PHP وانهيار الخادم بخطأ (OOM - Out Of Memory Fatal Error)، ناهيك عن تجميد خيط معالجة الويب (HTTP Worker) طوال فترة الاستعلام والحساب.

2. الحل المطبق (The Architectural Solution)

استجابةً للمتطلبات غير الوظيفية تم نقل معالجة البيانات الضخمة إلى وظيفة خلفية (Background Job) تقوم بجرد المبيعات اليومية ومعالجتها كـ "دفعات" (Chunks) لزيادة سرعة الأداء.

تم استبدال الواجهة المباشرة بواجهة (triggerDailySalesBatch) مهمتها الوحيدة هي جدولة المهمة ProcessDailySalesReportBatch وإرجاع استجابة سريعة للعميل، بينما تتولى العمال الخلفية (Queue Workers) المهمة الثقيلة بهدوء.

3. آلية العمل البرمجية (Engineering Mechanics): Chunking

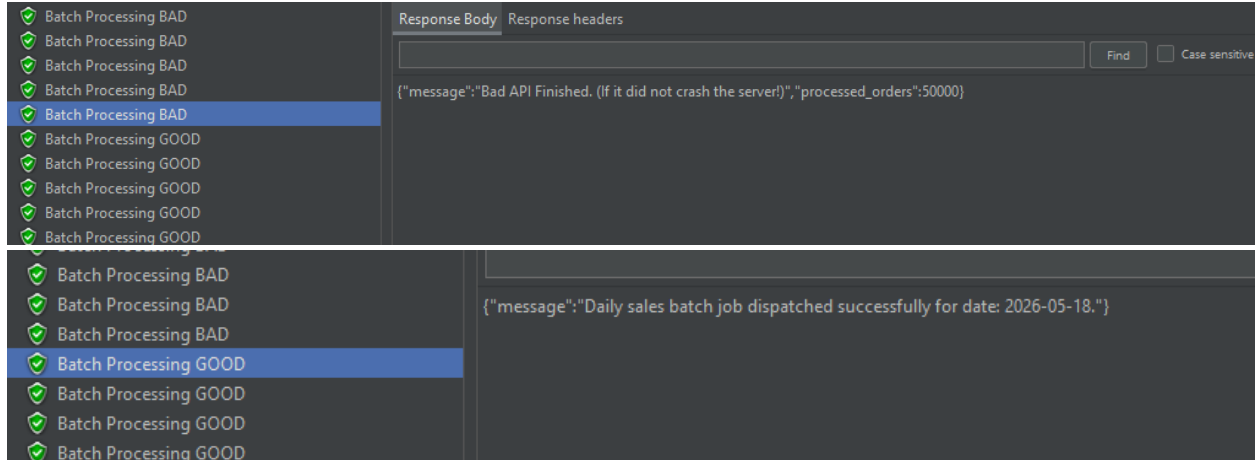
تم استبدال هذا الكود :

```
$date = $request->get('date', now()->toDateString());  
  
$orders = Order::whereDate('created_at', $date)  
    ->where('status', 'paid')  
    ->get();  
  
$salesBuffer = [];
```

بهذا ال worker:

```
$date = $request->get('date', now()->toDateString());  
  
ProcessDailySalesReportBatch::dispatch($date);
```

صور الاختبار على Jmeter:



صور الاختبار على grafana:



4. (جدول المقارنة الفنية (قبل وبعد التحسين

وجه المقارنة	الكود القديم Bad API (التحميل الكامل المتزامن)	الكود المطور Batch Job (المعالجة بالدفعات الخلفية)
استهلاك الذاكرة العشوائية ((RAM	مرتفع جداً ويزيد باستمرار (معرض للانهايار OOM عند البيانات الضخمة).	منخفض جداً وثابت (محدد بحجم الدفعة Chunk Size = 100).
زمن استجابة واجهة الـ API	بطيء جداً (ينتظر العميل حتى انتهاء قراءة وحساب كافة السجلات).	سريع جداً (استجابة فورية) (يتم توكيل المهمة للـ Queue).
الحمل على قاعدة البيانات	استعلام ضخم واحد يسبب قفل الجداول (Table Locks) لفترة طويلة.	استعلامات صغيرة وخفيفة متتالية لا تعيق العمليات المتزامنة الأخرى.
التوافق مع المتطلبات	يفشل في تلبية معايير الأنظمة القابلة للتوسع (Non-scalable).	يطبق مفهوم معالجة البيانات الضخمة بدفعات بشكل احترافي.

الطلب الخامس: توزيع الأحمال (Load Distribution))

1. توصيف الاختناق في العقدة المفردة (Single Node Bottleneck))

في الكود الأول checkoutSingleNode، يتم توجيه كامل حركة مرور الشبكة المتزامنة (100 خيط معالجة) إلى خادم ثابت واحد وهو Server_Node_1.

هذا النمط المعماري التقليدي يخلق نقطة فشل وحيدة (SPOF - Single Point of Failure) وينتهك معايير قابلية التوسع الأفقية (Horizontal Scalability)؛ حيث يصطدم الخادم سريعاً بسقف قدراته الفيزيائية (Hardware Wall) من حيث عدد خيوط المعالجة المتاحة في الـ Thread Pool، مما يؤدي إلى تزايد زمن الاستجابة (Latency) تدريجياً ليدخل النظام في حالة اختناق مادي كامل، وقد ينتهي الأمر بانهياره تحت الضغط العالي نتيجة عجز العقدة المفردة عن موازنة نمو الطلبات.

2. الحل واستراتيجية التوزيع (The Used Strategy))

تلبيةً لمتطلبات المشروع في محاكاة بيئات الحوسبة الموزعة، تم الانتقال في الدالة checkoutWithLoadDistribution إلى معمارية العنقود متعدد العقد (Multi-Node Cluster) المكون من عقدتين متطابقتين (Server_Node_1 و Server_Node_2).

استراتيجية التوزيع المستخدمة وتبريرها البرمجي: تم اعتماد خوارزمية التوزيع الدوري المنتظم (Round-Robin Load Balancing). تبرير اختيار هذه الاستراتيجية هندسياً يكمن في كون طلبات الشراء الواردة إلى الواجهة هي طلبات عديمة الحالة (Stateless Requests)، وتتمتع العقدتان بنفس المواصفات العتادية والبرمجية، مما يجعل خوارزمية Round-Robin الحل الأمثل والأسرع للتوزيع العادل للأحمال دون التسبب بأي عبء حوسبي إضافي (Zero Overhead) على خادم التوجيه.

3. آلية العمل البرمجية والمحاكاة (Implementation & Mechanics))

تتم عملية موازنة الأحمال برمجياً وبشكل دقيق عبر ثلاث خطوات متكاملة:

1. العداد الذري المشترك (Atomic Request Counter): يتم استدعاء الدالة Redis/Memcached). استخدام العداد هنا يتم بشكل "ذري" (Atomic Operation) لضمان عدم حدوث حالة تسابق (Race Condition) بين الخيوط المتنافسة أثناء تحديث قيمة العداد.
2. التوجيه الرياضي بالتناوب (Modulo Arithmetic Routing): يتم توجيه الطلبات ديناميكياً بالاعتماد على باقي قسمة قيمة العداد على عدد العقد المتاحة (في حالتنا عقدتان

صور من الكود :

```
public function checkoutWithLoadDistribution(Request $request)
{
    $startTime = microtime(true);

    try {
        $request->validate([
            'user_id' => 'required|exists:users,id',
            'store_id' => 'required|exists:stores,id',
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1'
        ]);

        $requestCounter = Cache::increment('load_balancer_request_count');
        $targetServer = ($requestCounter % 2 === 0) ? 'Server_Node_1' : 'Server_Node_2';

        $result = DB::transaction(function () use ($request) {
            $user = User::where('id', $request->user_id)->lockForUpdate()->firstOrFail();
            $storeProduct = StoreProduct::where('store_id', $request->store_id)
                ->where('product_id', $request->product_id)
                ->lockForUpdate()
                ->firstOrFail();

            $price = Product::find($request->product_id)->price;
            $total_price = $price * $request->quantity;

            if ($user->wallet_balance < $total_price || $storeProduct->quantity < $request->quantity)
                throw new \Exception("Insufficient stock or balance");

            $storeProduct->quantity -= $request->quantity;
            $storeProduct->save();

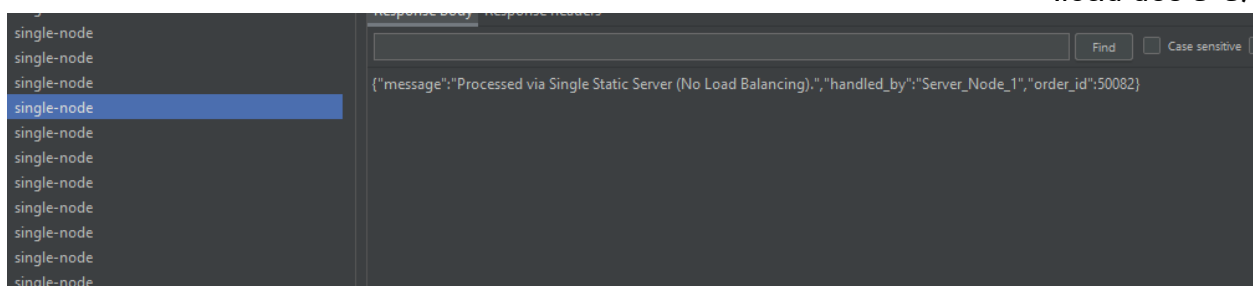
            $user->wallet_balance -= $total_price;
            $user->save();

            return Order::create([
                'user_id' => $user->id,
                'store_id' => $request->store_id,
                'total_price' => $total_price,
                'status' => 'paid'
            ]);
        });

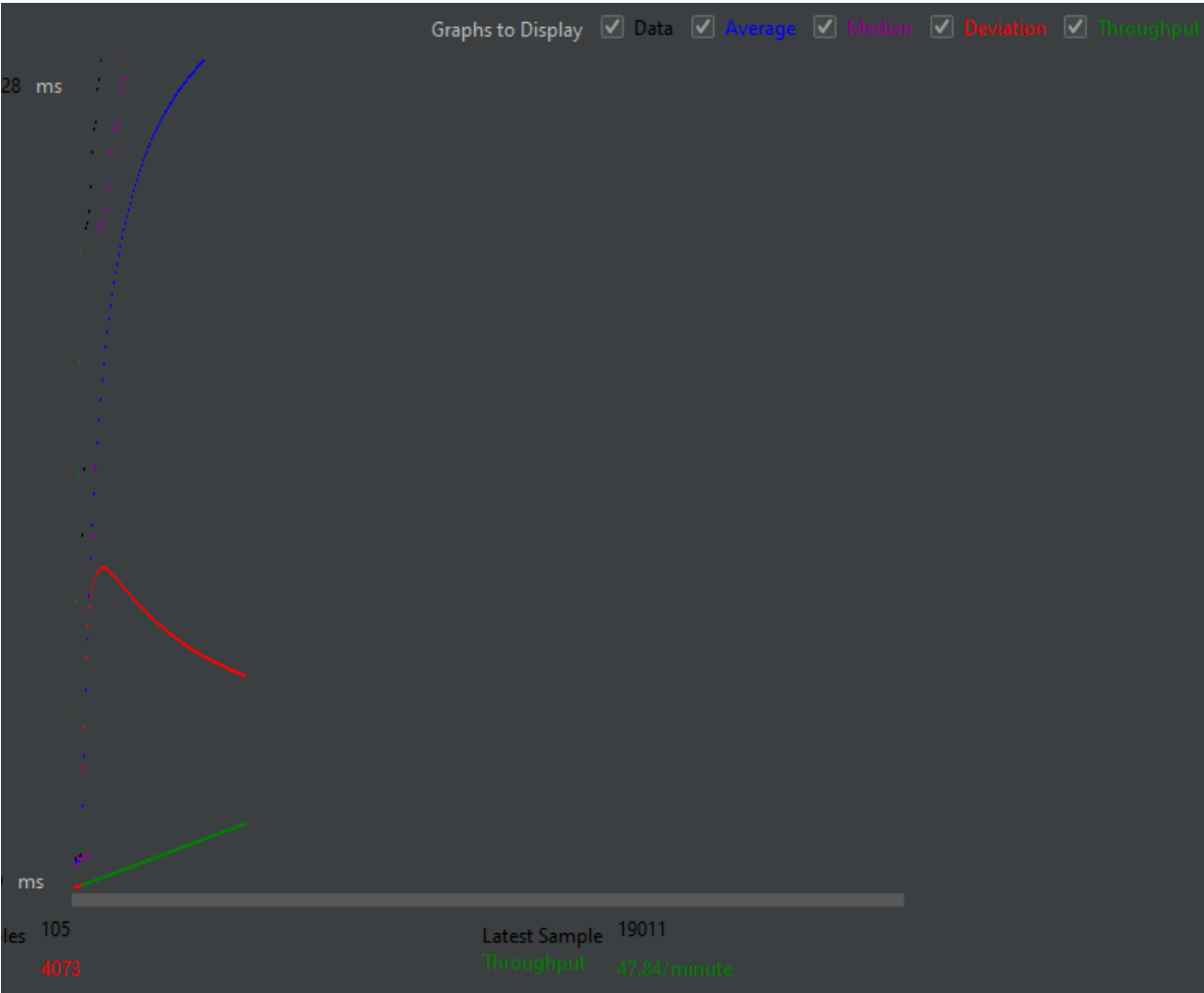
        usleep(30000);

        $duration = microtime(true) - $startTime;
    }
}
```

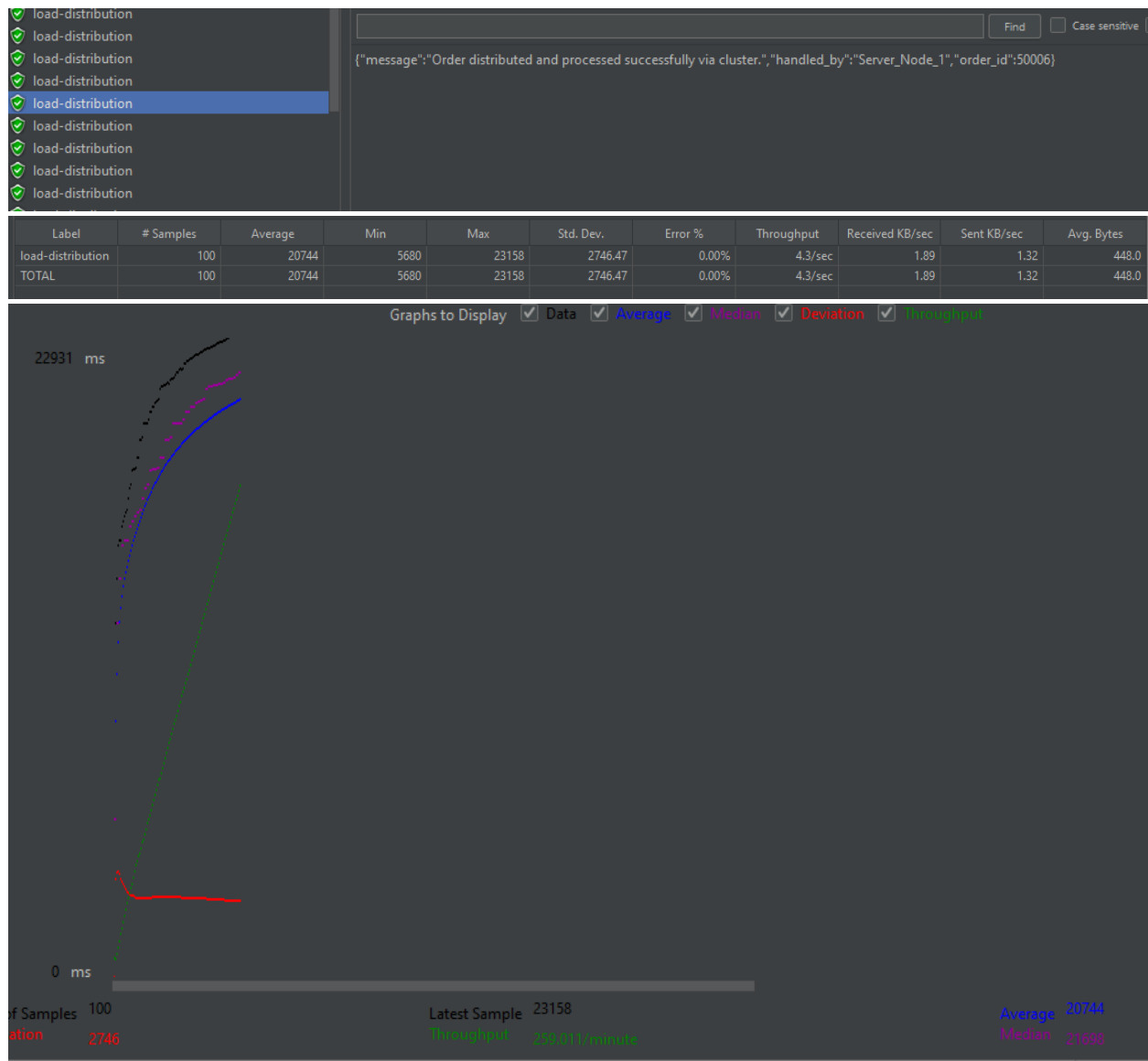
نتائج الاختبار على Jmeter :
قبل الـ load :



Filename:						
Label	# Samples	Average	Min	Max	Std. Dev.	
single-node	105	16553	456	19011	4073.73	
TOTAL	105	16553	456	19011	4073.73	



بعد التعديل :



4. جدول المقارنة الفنية (عقدة واحدة مقابل عنقود موزع)

وجه المقارنة الهندسية	معمارية العقدة المفردة checkoutSingleNode	معمارية التوزيع الموزع checkoutWithLoadDistribution
-----------------------	--	--

توزيع عادل متناظر: 50% لكل عقدة برمجياً.	100% من الضغط يتكدس على خادم واحد.	توزيع حركة المرور ((Traffic))
عالي؛ في حال سقوط عقدة، تستمر العقدة الأخرى بالعمل.	معدوم؛ سقوط الخادم يعني توقف المتجر بالكامل.	معامل وثوقية النظام ((Fault Tolerance))
مستقر ومنخفض، لتقاسم العبء الحوسبي بين مجمعي خيوط.	يتزايد طردياً بشكل حاد نتيجة اختناق خيوط المعالجة.	زمن الاستجابة تحت الضغط ((Latency))
توسع أفقي مرن (Horizontal) بإضافة عقد جديدة للمعادلة.	توسع عمودي محدود ومكلف ((Vertical)).	القدرة على التوسع ((Scalability))
تظهر العدادات منقسمة بالتساوي بين النود الأولى والثانية.	تظهر كافة الطلبات مسجلة تحت عقدة واحدة. Server_Node_1	طبيعة بيانات خط المراقبة metrics/